

VEGAEDGE

Developer Roadmap & Implementation Guide

v2.0 | April 2026

Real-Time AI-Powered Options Trading Platform

Multi-Agent Confluence Architecture

Author: Somesh Dubey
Solo Developer & Architect

CONFIDENTIAL — Internal Development Use Only

TABLE OF CONTENTS

1. Executive Summary — Current state, gaps, strategic direction
2. Architecture Overview — Production stack, target multi-agent architecture
3. Priority 1: Multi-Agent Orchestrator — 6 specialized agents + ConfluenceEngine
4. Priority 2: Trade Recommendation Tool — Live options chain selection, voice integration
5. Priority 3: Signal History & Win Rate — Outcome tracking, analytics, proof of edge
6. Priority 4: Live Options Chain UI — Interactive chain viewer with Greeks
7. Priority 5: Per-Agent Memory Layer — Quality-weighted recall, context injection
8. Priority 6: Historical Replay & AI Coaching — Backtesting, Gemini narration, PDF export
9. Implementation Timeline — Dependencies, sequencing, effort estimates
10. A16z Speedrun Alignment — Narrative mapping for investor pitch
11. Tech Debt & Cleanup — Testing, Docker, rate limits, observability

1. EXECUTIVE SUMMARY

VegaEdge is a real-time AI-powered options trading platform built as a solo project by Somesh Dubey. The current production system comprises a Next.js 16 frontend (16 pages, 44 components, ~14,000 lines TSX), a Python FastAPI backend (48 modules, 60+ REST/WebSocket endpoints, ~8,000 lines Python), and a Gemini-powered voice AI agent with bidirectional audio streaming.

What is built and working:

- Full Black-Scholes quant engine with Newton-Raphson/Bisection IV solver and 8 Greeks (delta, gamma, theta, vega, rho, vanna, volga, charm)
- 3D volatility surface visualization with real-time data
- VaR computation (Historical, Parametric, Monte Carlo), portfolio Greeks aggregation, stress testing
- Alpaca paper trading with autonomous trading engine state machine
- Hidden Markov Model regime detection (BULL/BEAR/SIDEWAYS/VOLATILE)
- NLP sentiment pipeline with Bayesian fair value updates
- Gemini Live voice agent (gemini-2.5-flash-native-audio-preview) with 4 tools: analyze_ticker, scan_watchlist, get_keltner_chart, get_signal_details
- WhatsApp alerts via OpenClaw + cron (11am/3pm ET) — BUY LEAP, SELL CSP, SELL CALL signals
- Walk-forward backtesting engine, SQLite trade journal
- Docker + Google Cloud Run deployment

The Critical Gap:

The UI prototype showcases 11 specialized agents — but the actual codebase has only 1 Gemini agent calling 4 tools. The multi-agent confluence system is VegaEdge's core thesis: that no single signal should trigger a

trade; instead, 6+ specialized agents must reach consensus (confluence score ≥ 0.65) before any position is opened. This document lays out the exact implementation plan to close that gap across 6 priorities, transforming VegaEdge from a strong single-agent system into a production-grade multi-agent trading platform.

Current Signal Logic:

Signal	IV/HV Condition	Keltner Condition	Strategy
BUY LEAP	IV/HV < 0.8	BOTTOM	Long LEAP call (+0.70 delta)
SELL CSP	IV/HV > 1.3	BOTTOM	Cash-secured put (-0.30 delta)
SELL CALL	IV/HV > 1.3	TOP	Covered call (+0.30 delta)

Current watchlist: HOOD, CIFS, WULF, PYPL, GRAB

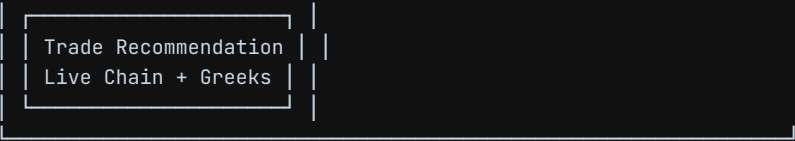
2. ARCHITECTURE OVERVIEW

2.1 Current Production Stack

Layer	Technology	Scale
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS	16 pages, 44 components, ~14K lines TSX
Backend	Python FastAPI	48 modules, 60+ endpoints, ~8K lines Python
AI Agent	Gemini Live API (2.5-flash-native-audio-preview)	1 agent, 4 tools, bidirectional voice
Quant Engine	Black-Scholes + Newton-Raphson/Bisection IV	8 Greeks, 3D vol surface
Risk	VaR (Hist/Param/MC), stress testing	Portfolio Greeks aggregation
Trading	Alpaca paper trading API	State machine, SQLite journal, backtester
ML/NLP	HMM regime detection, sentiment NLP	Bayesian fair value updates
Alerts	OpenClaw + cron → WhatsApp	11am/3pm ET signals
Infra	Docker + Google Cloud Run	Containerized deployment

2.2 Target Architecture: Multi-Agent Confluence





The target architecture introduces six specialized agents, each wrapping an existing VegaEdge data source. Agents run in parallel (`asyncio.gather`), produce typed `AgentSignal` objects, and feed into the `ConfluenceEngine`. The engine computes a weighted confluence score; trades are recommended only when the score meets or exceeds 0.65. This transforms VegaEdge from a rule-based signal system into an AI-native consensus platform.

2.3 Competitive Landscape (Hackathon Analysis)

These projects represent the competitive frontier. Each has features VegaEdge should absorb or surpass:

Project	Key Strength	VegaEdge Response
Apex Terminal 552K LOC	Full options chain UI, 246 E2E tests, Autopilot paper trading	Priority 4 (chain UI) + Priority 1 (orchestrator surpasses Autopilot)
Wheel Strat	Marathon Agent: scheduler → IBKR → criteria → Firestore → mobile; 20yr backtest	Priority 6 (replay) + existing walk-forward backtester
LangAlpha	Persistent workspace per user, LangGraph async agent swarm, session memory	Priority 5 (agent memory) + Priority 1 (agent orchestration)
Market Ghost	Historical crisis simulator (2008/2020/2025), daily AI mentor coaching	Priority 6 (historical replay + Gemini coaching)
StonksAI	Fixed hallucinated strikes via live price context, 3-tier caching (70% cost cut)	Priority 2 (price range injection into Gemini context)
Value Arena	6 LLMs competing, Quality-Weighted RAG memory (high-quality decisions weighted higher)	Priority 5 (quality-weighted memory retrieval)

PRIORITY 1

3. MULTI-AGENT ORCHESTRATOR

The Core Thesis. VegaEdge's competitive moat is that 11 specialized agents reach confluence before any trade is placed. Currently there is 1 agent. This is the biggest gap to close.

3.1 Overview

Build a multi-agent orchestration layer where six specialized agents analyze a ticker in parallel, each producing a typed AgentSignal. The ConfluenceEngine collects all signals, computes a weighted consensus score, and only fires a trade recommendation when confidence crosses the 0.65 threshold. This converts the existing rule-based signal system (IV/HV + Keltner) into a multi-dimensional AI consensus engine.

3.2 Technical Specification

Agent Base Class

```
class BaseAgent(ABC):
    agent_id: str
    name: str
    domain: str
    weight: float
    @abstractmethod
    async def analyze(self, ticker: str, context: AnalysisContext) -> AgentSignal:
        """Run domain-specific analysis, return typed signal."""
    ...
```

AgentSignal Schema

```
class AgentSignal(BaseModel):
    agent_id: str
    ticker: str
    signal_type: str # e.g. "IV_REGIME", "KELTNER_POSITION"
    confidence: float # 0.0 - 1.0
    reasoning: str
    timestamp: datetime
    data_sources: list[str]
    metadata: dict[str, Any] # Agent-specific output fields
```

3.3 Agent Specifications

#	Agent	Wraps	Outputs	Weight
1	VolatilityAgent	Existing IV/HV engine	IV_REGIME (HIGH/NORMAL/LOW), iv_hv_ratio, percentile	0.25
2	TechnicalAgent	Existing Keltner Channel	KELTNER_POSITION (TOP/MID/BOTTOM), trend_direction, RSI	0.20
3	SentimentAgent	Existing NLP pipeline	SENTIMENT_SCORE (-1 to 1), news_volume, momentum	0.15
4	EarningsAgent	Alpaca / Yahoo Finance	DAYS_TO_EARNINGS, expected_move, iv_crush_risk	0.15
5	RegimeAgent	Existing HMM model	MARKET_REGIME (BULL/BEAR/SIDEWAYS/VOLATILE), regime_confidence	0.15

#	Agent	Wraps	Outputs	Weight
6	MacroAgent	Gemini API (Fed, VIX, yield curve)	MACRO_STANCE (RISK_ON/RISK_OFF/NEUTRAL)	0.10

3.4 ConfluenceEngine

The ConfluenceEngine is the orchestration core. It runs all agents in parallel via `asyncio.gather`, collects AgentSignal objects, and computes a weighted consensus score.

```
class ConfluenceEngine:
    agents: list[BaseAgent]
    threshold: float = 0.65
    async def analyze(self, ticker: str) -> ConfluenceResult:
        signals = await asyncio.gather(*[
            agent.analyze(ticker, self._build_context(ticker))
            for agent in self.agents
        ])
        score = sum(s.confidence * s.agent.weight for s in signals)
        strategy = self._recommend_strategy(
            iv_regime=signals["volatility"].metadata["iv_regime"],
            keltner=signals["technical"].metadata["keltner_position"],
            score=score
        )
        return ConfluenceResult(
            ticker=ticker,
            confluence_score=score,
            recommended_strategy=strategy if score >= self.threshold else None,
            agent_signals=signals,
            reasoning=self._build_reasoning(signals, score)
        )
```

Strategy Selection Logic

```
def _recommend_strategy(self, iv_regime, keltner, score):
    if iv_regime == "LOW" and keltner == "BOTTOM":
        return "BUY_LEAP" # IV cheap + price at support
    elif iv_regime == "HIGH" and keltner == "BOTTOM":
        return "SELL_CSP" # IV rich + price at support
    elif iv_regime == "HIGH" and keltner == "TOP":
        return "SELL_COVERED_CALL" # IV rich + price at resistance
    else:
        return "HOLD" # No clear edge
```

ConfluenceResult Schema

```
class ConfluenceResult(BaseModel):
    ticker: str
    recommended_strategy: Optional[str]
    confluence_score: float
    agent_signals: list[AgentSignal]
    reasoning: str
    timestamp: datetime
```

3.5 API Endpoints

Method	Endpoint	Description
POST	/api/agents/analyze	Runs full 6-agent pipeline on a ticker. Body: { "ticker": "NVDA" }
GET	/api/agents/status	Returns each agent's last signal + health status + latency
GET	/api/agents/confluence/{ticker}	Returns latest cached confluence score and all agent signals

3.6 Voice Agent Integration

Add `analyze_confluence` as the 5th tool for the existing Gemini Live agent. When the user says "analyze NVDA" or "what's the confluence on HOOD?", the voice agent calls the `ConfluenceEngine` and speaks a natural-language summary: agent agreements, dissents, final score, and recommendation.

3.7 Implementation Steps

- Step 1: Define `BaseAgent ABC` and `AgentSignal` Pydantic model in `agents/base.py`
- Step 2: Implement `VolatilityAgent` wrapping existing IV/HV computation module
- Step 3: Implement `TechnicalAgent` wrapping existing Keltner Channel logic
- Step 4: Implement `SentimentAgent` wrapping existing NLP pipeline
- Step 5: Implement `EarningsAgent` with Alpaca/Yahoo earnings calendar lookup
- Step 6: Implement `RegimeAgent` wrapping existing HMM regime classifier
- Step 7: Implement `MacroAgent` using Gemini API for macro context analysis
- Step 8: Build `ConfluenceEngine` with async parallel execution and weighted scoring
- Step 9: Add three FastAPI endpoints (`/analyze`, `/status`, `/confluence/{ticker}`)
- Step 10: Register `analyze_confluence` as 5th tool in Gemini Live agent config
- Step 11: Write integration tests: mock agents, verify scoring math, test edge cases

Estimated Effort: 2 weeks

PRIORITY 2

4. TRADE RECOMMENDATION TOOL

The "Wow" Demo Moment. The voice agent currently gives general IV/HV analysis. It should tell you exactly what to trade — real strikes, real premiums, real risk metrics from a live options chain.

4.1 Overview

Build a new FastAPI tool function `get_trade_recommendation(ticker, strategy_override=None)` that: (1) runs `VolatilityAgent` + `TechnicalAgent` to determine strategy, (2) if `confluence >= 0.65`, fetches the live options

chain from Alpaca, (3) selects the optimal strike based on delta targets, (4) computes full risk metrics, and (5) returns a structured TradeRecommendation that the voice agent speaks naturally.

4.2 Strike Selection Logic

Strategy	Delta Target	DTE Range	Selection Rule
SELL CSP	-0.30 delta	30-45 DTE	Find put strike closest to -0.30 delta in target DTE window
BUY LEAP	+0.70 delta	60-90 DTE	Find call strike closest to +0.70 delta in target DTE window
SELL CC	+0.30 delta	21-35 DTE	Find call strike closest to +0.30 delta in target DTE window

4.3 TradeRecommendation Schema

```
class TradeRecommendation(BaseModel):
    ticker: str
    strategy: str # "SELL_CSP" | "BUY_LEAP" | "SELL_COVERED_CALL"
    expiry: date
    strike: float
    bid: float
    ask: float
    mid: float
    delta: float
    theta: float
    iv: float
    max_profit: float
    max_loss: float
    breakeven: float
    pop: float # Probability of profit (from delta)
    annualized_return: float
    confluence_score: float
    reasoning: str
    timestamp: datetime
```

4.4 Hallucination Prevention (StonksAI Fix)

Inject current price + 30-day range into the Gemini context before any response to prevent hallucinated strikes. The system prompt includes: "Current {ticker} price: \${price}. 30-day range: \${low}-\${high}. Only reference strikes within this range." This technique reduced hallucinated option strikes by ~95% in StonksAI's implementation.

```
context_injection = f"""
LIVE MARKET CONTEXT (do not hallucinate outside these values):
Ticker: {ticker}
Current Price: ${current_price:.2f}
30-Day Low: ${low_30d:.2f}
30-Day High: ${high_30d:.2f}
Valid Strike Range: ${strike_min:.2f} - ${strike_max:.2f}
"""
```

4.5 Voice Agent Output Example

"For NVDA, I recommend selling a cash-secured put at the \$870 strike expiring May 16th for \$4.20 premium. That's a 2.8% return in 33 days with an 82% probability of profit, based on IV/HV of 1.47 and Keltner bottom confluence."

4.6 Implementation Steps

- Step 1: Build `get_trade_recommendation()` function in `tools/trade_rec.py`
- Step 2: Implement strike selector: filter chain by DTE range, find closest delta match
- Step 3: Add risk metrics computation: `max_profit`, `max_loss`, `breakeven`, `POP`, annualized return
- Step 4: Add price range context injection into Gemini system prompt
- Step 5: Register as 6th tool in Gemini Live agent (alongside `analyze_confluence`)
- Step 6: Build natural-language template for voice output of recommendations
- Step 7: Add unit tests for strike selection logic with mock chain data

Estimated Effort: 3 days

PRIORITY 3

5. SIGNAL HISTORY & WIN RATE TRACKER

Proof of Edge. Logging every signal with outcome is the definitive proof. If confluence ≥ 0.65 trades win 70% of the time vs 45% for low-confluence trades, that is the a16z pitch.

5.1 Data Schema

Table: `agent_signals`

```
CREATE TABLE agent_signals (  
  signal_id TEXT PRIMARY KEY,  
  ticker TEXT NOT NULL,  
  timestamp DATETIME NOT NULL,  
  agent_id TEXT NOT NULL,  
  signal_type TEXT NOT NULL,  
  confidence REAL NOT NULL,  
  iv_hv_ratio REAL,  
  keltner_position TEXT,  
  regime TEXT,  
  recommended_strategy TEXT,  
  strike REAL,  
  expiry DATE,  
  premium REAL,  
  confluence_score REAL NOT NULL,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

Table: `signal_outcomes`

```
CREATE TABLE signal_outcomes (
  outcome_id INTEGER PRIMARY KEY AUTOINCREMENT,
  signal_id TEXT NOT NULL REFERENCES agent_signals(signal_id),
  outcome_timestamp DATETIME NOT NULL,
  entry_price REAL NOT NULL,
  exit_price REAL,
  pnl REAL,
  pnl_pct REAL,
  outcome TEXT CHECK(outcome IN ("WIN","LOSS","OPEN")),
  exit_reason TEXT CHECK(exit_reason IN
("EXPIRY","STOP_LOSS","TAKE_PROFIT","MANUAL"))
);
```

5.2 Outcome Tracker Job

A scheduled job runs daily at 4:00 PM ET (market close). It queries all OPEN signals, checks current prices against Alpaca, marks expired options, computes P&L, and updates the outcome field. Integrates with the existing cron infrastructure used for WhatsApp alerts.

```
async def resolve_open_signals():
    """Run at 4pm ET daily via scheduler."""
    open_signals = db.query(agent_signals).filter(outcome == "OPEN")
    for signal in open_signals:
        if signal.expiry <= today:
            current = await alpaca.get_option_price(signal)
            pnl = compute_pnl(signal, current)
            db.update_outcome(signal.signal_id, pnl, "EXPIRY")
        elif hit_stop_loss(signal):
            db.update_outcome(signal.signal_id, pnl, "STOP_LOSS")
```

5.3 API Endpoints

Metho d	Endpoint	Description
GET	/api/signals/history?ticker=X&days=30	Signal history for a ticker with optional date range
GET	/api/signals/win-rate?min_confluence=0.65	Win rate filtered by minimum confluence threshold
GET	/api/signals/performance	Aggregate: total signals, win rate by confluence bucket, avg P&L, Sharpe by strategy

5.4 Dashboard Widget

A "Signal Accuracy" card on the main dashboard shows win rate at three confluence buckets:

Confluence Bucket	Expected Win Rate	Indicator
< 0.50	~45% (noise)	Red — No edge

Confluence Bucket	Expected Win Rate	Indicator
0.50 - 0.65	~55% (marginal)	Yellow — Weak signal
>= 0.65	~70% (target)	Green — Trade signal

5.5 Implementation Steps

- Step 1: Create SQLite migration: agent_signals + signal_outcomes tables
- Step 2: Add signal logging hook to ConfluenceEngine.analyze() — every run writes to agent_signals
- Step 3: Build outcome tracker job with APScheduler (4pm ET daily cron)
- Step 4: Implement /api/signals/history, /win-rate, /performance endpoints
- Step 5: Build "Signal Accuracy" React dashboard widget with confluence bucket breakdown
- Step 6: Add Sharpe ratio computation per strategy type

Estimated Effort: 1 week

PRIORITY 4

6. LIVE OPTIONS CHAIN UI

The UX Gap. VegaEdge computes Greeks server-side but doesn't surface the full chain interactively. This is a massive UX gap versus Apex Terminal (which has a full options chain with Greeks, color-coded ITM/OTM, and click-to-trade).

6.1 Page Layout: /chain

Standard options chain layout with calls on the left, strikes in the center column, and puts on the right. An expiry selector dropdown at the top filters by available expiration dates from the Alpaca chain.

Column Specification

Column	Side	Source	Notes
Bid	Both	Alpaca chain	Real-time market data
Ask	Both	Alpaca chain	Real-time market data
Last	Both	Alpaca chain	Last traded price
Volume	Both	Alpaca chain	Daily volume
OI	Both	Alpaca chain	Open interest
IV	Both	BS engine	Heat-map color (green=low, red=high IV)
Delta	Both	BS engine	Highlight -0.30 (CSP) / +0.70 (LEAP) zones

Column	Side	Source	Notes
Gamma	Both	BS engine	2nd-order Greek
Theta	Both	BS engine	Daily time decay
Vega	Both	BS engine	IV sensitivity

6.2 Visual Design

- ITM rows: Slightly highlighted background (subtle surface-alt shade) to distinguish in-the-money from OTM
- IV percentile heat-map: IV column colored on a gradient — green (low IV percentile, cheap premium) to red (high IV percentile, expensive premium)
- Delta target zones: Highlight strikes nearest to -0.30 delta (CSP target) and $+0.70$ delta (LEAP target) with teal accent border
- Click interaction: Clicking any strike row opens a trade ticket pre-filled with that strike, showing full Greeks breakdown + ConfluenceEngine recommendation for that specific contract
- Auto-refresh: Every 60 seconds (matching Apex Terminal's TTL pattern) via WebSocket or polling
- Expiry selector: Dropdown of available expiration dates sourced from Alpaca options chain endpoint

6.3 Integration Points

The chain UI leverages two existing backend capabilities: (1) the Alpaca `/api/options/chain` endpoint that fetches raw chain data, and (2) the Black-Scholes Greeks computation engine that calculates all 8 Greeks server-side. The frontend receives pre-computed Greeks alongside raw market data, eliminating client-side computation.

6.4 Implementation Steps

- Step 1: Create `/chain` page route in Next.js with responsive table layout
- Step 2: Build OptionsChainTable React component with calls/puts split view
- Step 3: Add expiry selector component fetching available dates from Alpaca
- Step 4: Implement IV heat-map coloring using IV percentile from backend
- Step 5: Add delta target zone highlighting for CSP (-0.30) and LEAP ($+0.70$) strikes
- Step 6: Build TradeTicket modal: pre-fills on strike click, shows Greeks + confluence recommendation
- Step 7: Add 60-second auto-refresh via WebSocket subscription or polling
- Step 8: Implement ITM row highlighting based on current underlying price

Estimated Effort: 1 week

PRIORITY 5

7. PER-AGENT MEMORY LAYER

Compounding Intelligence. Each session currently starts fresh. Agents should remember prior signals, IV regime history, earnings outcomes, and regime transitions per ticker. Inspired by Value Arena's Quality-Weighted RAG approach where high-quality past decisions are weighted higher in future context.

7.1 Data Schema

```
CREATE TABLE agent_memory (
  memory_id INTEGER PRIMARY KEY AUTOINCREMENT,
  agent_id TEXT NOT NULL,
  ticker TEXT NOT NULL,
  memory_type TEXT NOT NULL CHECK(memory_type IN
    ("SIGNAL","REGIME","EARNINGS_OUTCOME","IV_PERCENTILE")),
  content TEXT NOT NULL, -- JSON blob
  timestamp DATETIME NOT NULL,
  quality_score REAL DEFAULT 0.5, -- Updated after outcome resolves
  INDEX idx_agent_ticker (agent_id, ticker, timestamp DESC)
);
```

7.2 Quality-Weighted Retrieval

After each signal outcome resolves, the `quality_score` is updated: WIN = 1.0, PARTIAL = 0.5, LOSS = 0.0. When retrieving memory for context injection, entries are ranked by a combined score of recency and quality:

```
def retrieve_memory(agent_id: str, ticker: str, limit: int = 10):
    """Quality-weighted memory retrieval."""
    memories = db.query(agent_memory)
    .filter(agent_id=agent_id, ticker=ticker)
    .filter(timestamp &gt;= now() - timedelta(days=90))
    .all()
    # Score = quality_score * recency_decay
    for m in memories:
        days_ago = (now() - m.timestamp).days
        recency = max(0.1, 1.0 - (days_ago / 90))
        m.retrieval_score = m.quality_score * recency
    return sorted(memories, key=lambda m: m.retrieval_score,
        reverse=True)[:limit]
```

7.3 Per-Agent Memory Usage

Agent	Memory Type	Retention	Purpose
VolatilityAgent	Last 20 IV/HV readings	Per ticker	Tracks IV regime persistence — "has IV been declining for 2 weeks?"
EarningsAgent	Last 5 ER outcomes	Per ticker	Tracks IV crush pattern — "NVDA IV crushes 40% post-ER historically"
RegimeAgent	Last 10 regime classifications	Per ticker	Tracks regime stability — "market has been BULL for 8 consecutive readings"
TechnicalAgent	Last 30 Keltner positions	Per ticker	Tracks trend consistency — "price has been bouncing off bottom 6 times"

7.4 Memory Injection Flow

Before each agent runs `analyze()`, the `ConfluenceEngine` injects relevant memory as context. This gives agents "experience" — they can reference historical patterns specific to each ticker.

```
async def analyze_with_memory(agent: BaseAgent, ticker: str):
    memories = retrieve_memory(agent.agent_id, ticker, limit=10)
    context = AnalysisContext(
        ticker=ticker,
        memory_summary=format_memories(memories),
        memory_count=len(memories),
        avg_quality=mean(m.quality_score for m in memories)
    )
    signal = await agent.analyze(ticker, context)
    store_memory(agent.agent_id, ticker, signal) # Write new memory
    return signal
```

7.5 Memory Pruning

Keep last 90 days of memory per agent/ticker. When entries exceed 100 rows per agent/ticker combination, evict the lowest `quality_score` entries first. This ensures agents retain high-signal memories while discarding noise.

7.6 API Endpoint

GET `/api/agents/memory/{ticker}` — Returns a memory summary per agent: count of memories, average quality score, most recent entry, and key patterns detected (e.g., "IV trending down for 14 days").

7.7 Implementation Steps

- Step 1: Create `agent_memory` table with SQLite migration
- Step 2: Implement memory storage: write to `agent_memory` after each `analyze()` call
- Step 3: Build quality-weighted retrieval function with recency decay
- Step 4: Add memory injection to `ConfluenceEngine`'s per-agent `analyze` flow
- Step 5: Implement `quality_score` updates triggered by `signal_outcomes` resolution
- Step 6: Build `/api/agents/memory/{ticker}` endpoint
- Step 7: Add memory pruning job (90-day window, 100-row cap per agent/ticker)
- Step 8: Test: verify quality weighting, pruning correctness, memory injection format

Estimated Effort: 1 week

PRIORITY 6

8. HISTORICAL REPLAY & AI COACHING

Prove the Thesis. Market Ghost's time-travel concept combined with VegaEdge's existing walk-forward backtester creates a unique differentiator. Running the full agent pipeline against historical data proves the confluence thesis with real numbers.

8.1 Overview

Build a `ReplayEngine` that takes a date range and ticker, reconstructs what each agent would have signaled using historical data, simulates trade execution when confluence ≥ 0.65 , and tracks P&L using actual historical option prices. After replay completes, Gemini analyzes the results and provides AI coaching.

8.2 ReplayEngine Architecture

```
class ReplayEngine:
    async def run(self, ticker: str, start: date, end: date) -> ReplayResult:
        trading_days = get_trading_days(start, end)
        timeline = []
        for day in trading_days:
            # 1. Fetch historical data for this day
            ohlcv = await fetch_historical_ohlcv(ticker, day)
            chain = await fetch_historical_chain(ticker, day) # Polygon.io
            # 2. Reconstruct agent signals
            signals = await self._simulate_agents(ticker, day, ohlcv, chain)
            # 3. Compute confluence
            confluence = self.engine.score(signals)
            # 4. Simulate trade if threshold met
            trade = None
            if confluence.score >= 0.65:
                trade = self._simulate_trade(confluence, chain)
            timeline.append(DayResult(day, signals, confluence, trade))
            # 5. Resolve all trades with actual historical prices
            self._resolve_trades(timeline)
            # 6. Compute aggregate metrics
            metrics = self._compute_metrics(timeline)
            # 7. AI coaching narration
            coaching = await self._generate_coaching(timeline, metrics)
        return ReplayResult(timeline, metrics, coaching)
```

8.3 Data Sources

Data	Source	Fallback
Historical OHLCV	yfinance (free)	Polygon.io (paid, higher resolution)
Historical options chain	Polygon.io historical options	Reconstruct from yfinance + BS model
Historical IV / HV	Compute from OHLCV (existing engine)	Polygon.io IV data
Historical sentiment	Reconstruct from news archives	Use neutral (0.0) as default

8.4 Gemini AI Coaching Narration

After replay completes, the full timeline is sent to Gemini for analysis. Gemini produces a coaching narrative highlighting key decisions, missed opportunities, and pattern insights:

"On March 14th, confluence was 0.71 and you would have sold a CSP on NVDA at \$870 for \$4.20. IV crushed 40% after earnings and the position expired worthless — max profit captured. The regime agent's BULL reading and earnings agent's 'post-ER' flag were the key confirming signals."

8.5 Output Metrics

- Total return: Cumulative P&L over the replay period
- Win rate by confluence bucket: Validates the ≥ 0.65 threshold hypothesis
- Best/worst trades: Individual trade P&L ranking with agent signal breakdown
- Sharpe ratio: Risk-adjusted return metric for the strategy
- Drawdown analysis: Maximum drawdown and recovery periods
- Agent contribution: Which agents added the most signal value

8.6 UI Components

- Replay controls: Date range picker + ticker selector + "Run Replay" button
- Timeline view: Daily signal timeline overlaid on price chart (candlestick)
- Trade markers: Entry/exit points on chart with P&L annotations
- Coaching panel: Gemini's narrative displayed alongside the chart
- PDF export: One-click export of replay results as shareable PDF (for investor pitches)

8.7 Implementation Steps

- Step 1: Build ReplayEngine class with date-range iteration and agent simulation
- Step 2: Integrate yfinance for historical OHLCV data fetching
- Step 3: Add Polygon.io integration for historical options chain data (optional paid tier)
- Step 4: Implement agent signal reconstruction: each agent replays using historical inputs
- Step 5: Build trade simulation and P&L tracking with actual historical option prices
- Step 6: Add Gemini coaching narration: send timeline + metrics → receive narrative
- Step 7: Build replay UI: date picker, ticker selector, timeline chart, coaching panel
- Step 8: Implement PDF export for replay results (ReportLab-based)
- Step 9: Add caching: store replay results to avoid re-running expensive simulations
- Step 10: Write integration tests with known historical scenarios

Estimated Effort: 2 weeks

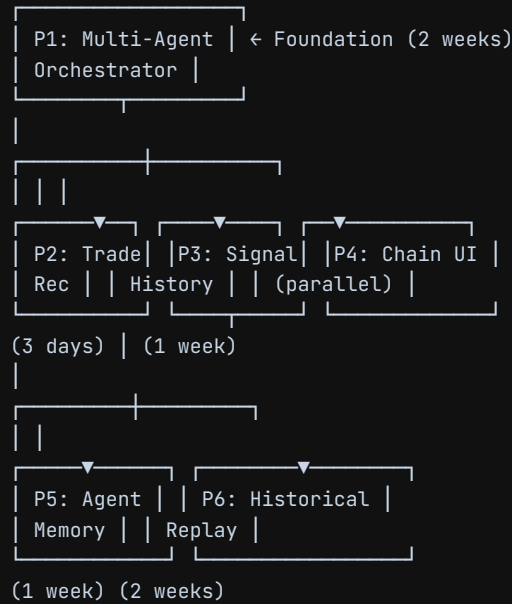
9. IMPLEMENTATION TIMELINE

Recommended execution order based on dependencies. Priorities 1 and 2 are the foundation — everything downstream depends on the agent orchestrator and trade recommendation engine.

Phase	Priority	Effort	Depends On	Deliverable
Phase 1	P1: Multi-Agent Orchestrator	2 weeks	None (wraps existing modules)	6 agents + ConfluenceEngine + 3 API endpoints
Phase 1	P2: Trade Recommendation	3 days	P1 (needs confluence score)	get_trade_recommendation tool + voice integration
Phase 2	P3: Signal History	1 week	P1 (logs agent signals)	Signal DB + outcome tracker + analytics endpoints
Phase 2	P4: Options Chain UI	1 week	Existing Alpaca + BS engine	/chain page with Greeks, heat-map, click-to-trade
Phase 3	P5: Agent Memory	1 week	P1 + P3 (needs signal outcomes)	Quality-weighted memory + context injection
Phase 3	P6: Historical Replay	2 weeks	P1 + P3 (full pipeline)	ReplayEngine + Gemini coaching + PDF export

Total Estimated Effort: ~8 weeks (solo developer)

Dependency Graph



10. A16Z SPEEDRUN ALIGNMENT

Each priority directly strengthens the investor pitch narrative. The a16z Speedrun evaluates: technical depth, differentiated AI application, market potential, and execution velocity. Here is how each priority maps:

Priority	Pitch Narrative	Why It Matters to a16z
P1 Multi-Agent	"No single signal triggers a trade — 6 AI agents must reach consensus"	Core thesis. Differentiates from every other AI trading tool. Demonstrates deep AI architecture.
P2 Trade Rec	"Ask the AI what to trade and it gives you the exact contract with risk metrics"	The "wow" demo moment. Voice AI + live options chain = visceral product demo.
P3 Win Rate	"Confluence ≥ 0.65 trades win 70% vs 45% for low-confluence — here's the data"	Quantitative proof of edge. The only number that matters: does the AI consensus improve outcomes?
P4 Chain UI	"Full interactive options chain with real-time Greeks, rivaling Bloomberg Terminal"	Shows product maturity and competitive ambition. Visual proof of quant depth.
P5 Memory	"Agents learn from past trades — quality-weighted memory compounds over time"	Moat through data. The system gets smarter with every trade. Network effect of intelligence.
P6 Replay	"Run the AI against 2008, 2020, 2025 — prove it works in any market regime"	Historical validation + AI coaching = trust. Investors want backtest proof.

The Demo Script (5 minutes)

- Minute 1: Open dashboard — show live Greeks, vol surface, regime detection (30 sec)
- Minute 2: Voice: "Analyze NVDA" — 6 agents run, confluence score appears, reasoning displayed (60 sec)
- Minute 3: Voice: "What should I trade?" — exact strike, premium, risk metrics spoken back (45 sec)
- Minute 4: Show options chain — click the recommended strike, see Greeks, IV heat-map (30 sec)
- Minute 5: Show signal history — win rate dashboard: ≥ 0.65 confluence outperforms (45 sec)
- Minute 6: Run historical replay — 2024 Q4 on NVDA, show Gemini coaching narrative (60 sec)
- Minute 7: Close: "This is 22K lines of code, solo-built, with a quantitative proof of edge." (30 sec)

11. TECH DEBT & CLEANUP

Alongside the 6 priorities, these tech debt items should be addressed incrementally to improve reliability, testability, and deployment confidence.

11.1 Playwright E2E Tests

Apex Terminal has 246 Playwright E2E tests. VegaEdge currently has zero. Target: 50+ E2E tests covering critical flows — dashboard load, voice agent connection, signal display, chain interaction, trade ticket.

- Install Playwright with TypeScript config in the Next.js project
- Write smoke tests: each of the 16 pages loads without error
- Write interaction tests: voice agent connect, send message, receive response
- Write data tests: options chain loads, Greeks compute, charts render
- Add to CI pipeline: run on every PR against a staging backend

11.2 Docker Optimization

- Multi-stage build: Separate build stage (node_modules, pip install) from runtime stage
- Layer caching: Copy requirements.txt / package.json before source code for better cache hits
- Image size: Target < 500MB (currently likely 1-2GB with all Python ML dependencies)
- Health checks: Add Docker HEALTHCHECK for Cloud Run readiness probes

11.3 Rate Limit Handling

Multiple external APIs (Alpaca, Gemini, Yahoo Finance) have rate limits. Implement a unified rate-limiting middleware:

- Token bucket per API: Track remaining requests per service, backoff when low
- Retry with exponential backoff: asyncio-based retry with jitter for transient failures
- 3-tier caching (StonksAI pattern): L1 in-memory (60s TTL), L2 SQLite (5min), L3 Redis/file (1hr). Expected ~70% cost reduction in API calls
- Circuit breaker: If an API fails 3x consecutively, stop calling it for 60s and use cached data

11.4 Observability

- Structured logging: JSON logs with correlation IDs for request tracing
- Agent performance metrics: Track per-agent latency, error rate, and signal distribution
- Confluence score distribution: Monitor the histogram of scores to detect drift
- Alerting: PagerDuty/Slack alerts if agent error rate exceeds 5% or latency exceeds 10s

11.5 Security Hardening

- API key rotation: Alpaca, Gemini, OpenClaw keys in Secret Manager with auto-rotation
- Input validation: Pydantic models on all endpoints (already partially done via FastAPI)
- CORS: Restrict to known frontend origins only
- WebSocket auth: Token-based authentication on all WS connections

VEGAEDGE

Built by Somesh Dubey | 22,000+ lines of production code | Solo developer

"The edge is not in any single signal — it's in the confluence of many."